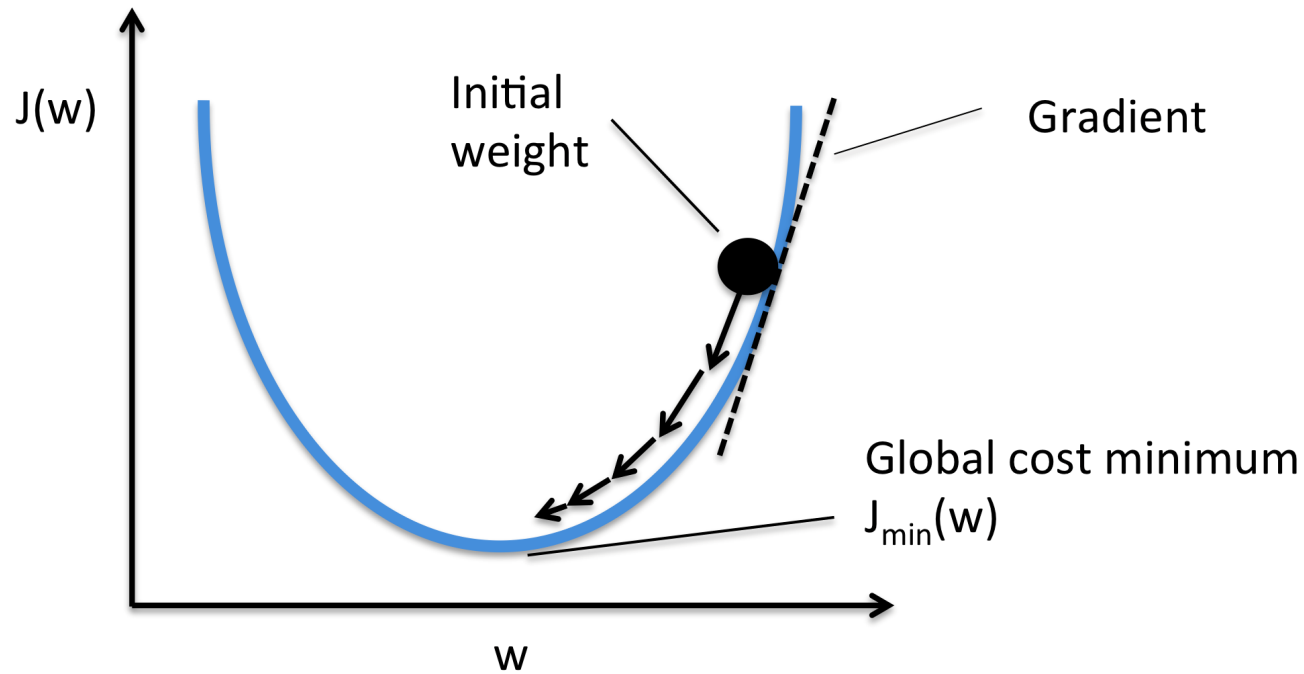


Empirical Explanation of Gradient Descent



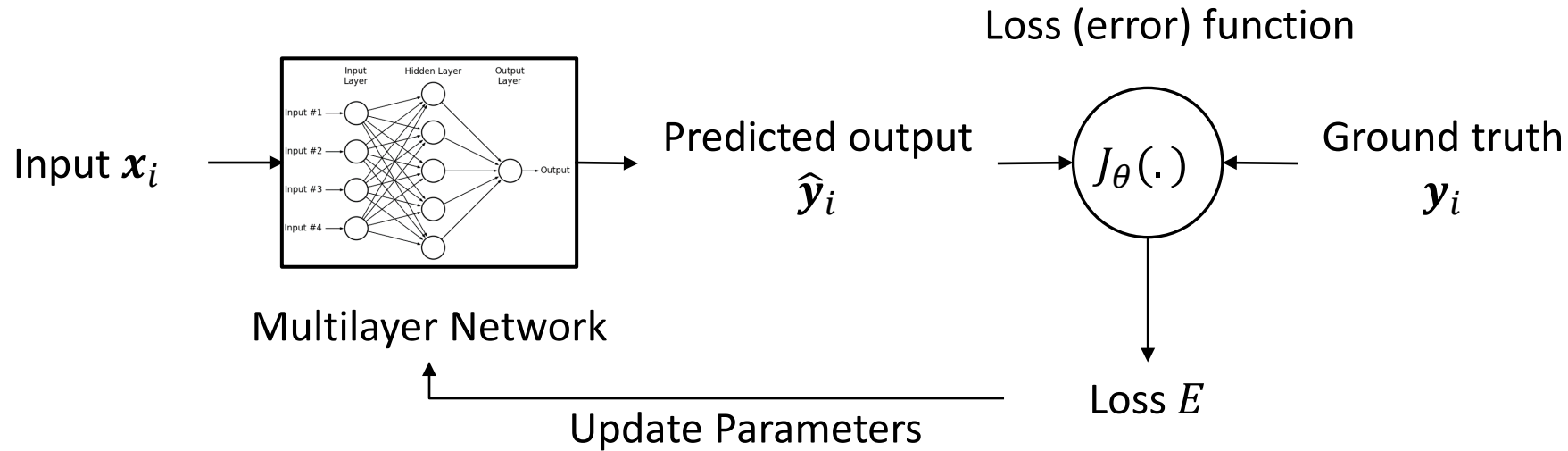
$$\theta_j = \theta_j - \eta \cdot \frac{\partial}{\partial \theta_j} J_{\theta}$$

↓

η is the **learning rate**, which controls the 'step size' of the optimization

- ❑ Our objective is to minimize the loss function J , which is a function of the model parameters
- ❑ A gradient measures how much the output of a function changes if you change the inputs a little bit
- ❑ We update the parameters, based on their gradients, so that the loss function is going 'downhill'

Learning NN Parameters



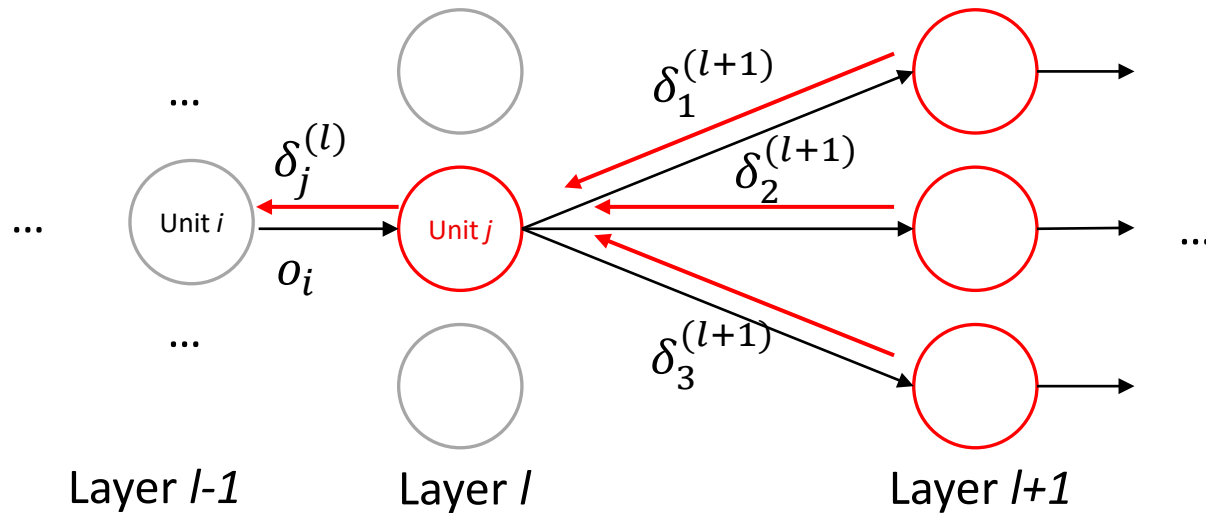
❑ Gradient Descent Algorithm

❑ **Input:** Training sample x_i and its label y_i

1. **Feed Forward:** Get prediction $\hat{y}_i = \text{MLP}(x_i)$, and loss $E = J(\hat{y}_i, y_i)$
2. **Compute Gradient:** For each parameter θ_j (weights, bias), compute its gradient $\frac{\partial}{\partial \theta_j} J_\theta$
3. **Update Parameter:** $\theta_j = \theta_j - \eta \cdot \frac{\partial}{\partial \theta_j} J_\theta$

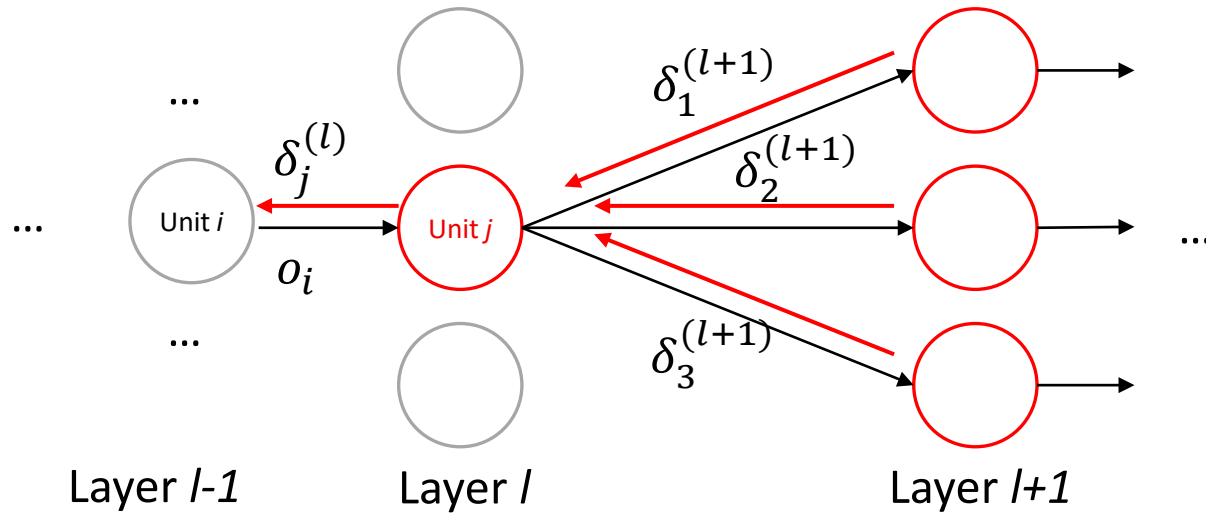
↑
Explained later

Gradient Computation: Backpropagation



- The gradient of w_{ij} in the l th layer (corresponding to unit j in layer l , connected to unit i in layer $l-1$) is a function of
 - All 'error' terms from layer $l+1$, $\delta_k^{(l+1)}$ -- An auxiliary term for computation, not to be confused with gradients
 - Output from unit i in layer $l-1$ (input to unit j in layer l) -- Can be stored at the feed forward phase of computation

Gradient Computation: Backpropagation



- The 'error' terms $\delta_j^{(l)}$ is a function of
 - All $\delta_k^{(l+1)}$ in the layer $l+1$, if layer l is a hidden layer
 - The overall loss function value, if layer l is the output layer
- We can compute the error at the output, and distribute backwards throughout the network's layers (backpropagation)